

CMSE 801 - Day 12

Agent-based Modeling

Oct 12, 2022

General Course Announcements

- The midterm exam is next week!
- Homework #3 will be released today

Questions/comments from Day 12 PCA

- I'm confused as to why we loop over the rows and the columns in 2D arrays? Would just one or the other go over every index?
- need more help with indexing rows and column
- For the for loop, we take the i and the j values like so: for i in range(zeros.shape[0]): So is the [0] going to always be the rows? And the [1] is the columns? is it the i and j that determines that? Also explain the difference between the i and j perhaps?
- what is x_indexes[myarray == 1] doing
- I just wrote a easier code to finish the task than the instruction, can I use it instead of following the instruction?
- If the random function is "pseudo-random" based on the seed, are there some seeds that can get "non-random" numbers? Like we can get 10 same numbers at the beginning.
- I did not know how to do the last part.
- I think I need more examples of the agent-based models to better understand them!
- I need to work with a few more ABMs to fully grasp them, but I believe that will be coming in the next few classes
- I feel like this class took a big step with out me.
- Thank you! I feel like I am learning more than I have in years / Nothing at the moment, thanks!

```
In [1]: import numpy as np
```

```
In [2]: np.random.seed(123)
        for i in range(5):
            print(np.random.random())
```

```
0.6964691855978616
0.28613933495037946
0.2268514535642031
0.5513147690828912
0.7194689697855631
```

```
In [3]: for i in range(5):
```

```
print(np.random.random())
```

```
0.42310646012446096
0.9807641983846155
0.6848297385848633
0.48093190148436094
0.3921175181941505
```

```
In [4]: x_range = np.linspace(0, 1, 3)
y_range = np.linspace(0, 0.5, 5)
x_indexes, y_indexes = np.meshgrid(x_range, y_range)
print(x_range)
print(y_range)
print(x_indexes)
print(y_indexes)
```

```
[0.  0.5  1. ]
[0.    0.125 0.25  0.375 0.5   ]
[[0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]]
[[0.    0.    0.    ]
 [0.125 0.125 0.125]
 [0.25  0.25  0.25 ]
 [0.375 0.375 0.375]
 [0.5   0.5   0.5   ]]
```

```
In [5]: array = np.random.rand(5,3)
print(array)
print('---')
print(x_indexes)
print('---')
print(x_indexes[array>0.3])
```

```
[[0.34317802 0.72904971 0.43857224]
 [0.0596779  0.39804426 0.73799541]
 [0.18249173 0.17545176 0.53155137]
 [0.53182759 0.63440096 0.84943179]
 [0.72445532 0.61102351 0.72244338]]
---
[[0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]
 [0.  0.5  1. ]]
---
[0.  0.5  1.  0.5  1.  1.  0.  0.5  1.  0.  0.5  1. ]
```

```
In [6]: def create_forest(N, ftrestart):

    # Initialize a 2D array full of zeros using np.zeros
    forest = np.zeros((N,N))

    # Loop through the 2D array to initialize the trees
    for i in range(N):
        for j in range(N):
            if np.random.random() < ftrestart:
                forest[i,j] = 1

    forest[:,0] = 2
```

```
return forest
```

```
In [7]: forest_array = create_forest(5, .3)  
print(forest_array)
```

```
[[2. 0. 1. 1. 0.]  
 [2. 0. 0. 0. 0.]  
 [2. 0. 0. 0. 0.]  
 [2. 1. 0. 0. 0.]  
 [2. 0. 0. 0. 0.]]
```